数字逻辑与微处理器

主 讲：何学辉
电子工程学院电子所
Email： xhhe@mail.xidian.edu.cn
Mobile：13992859217

课程背景介绍

数字逻辑与系统设计+微机原理与接口设计之间关系

数字电路

- **数制与编码**：各种常用数制及其相互转换，BCD码，有符号数等
- **逻辑代数基础**：基本逻辑运算及逻辑门，表达式，卡诺图等
- **集成逻辑门**：逻辑门输入/输出特性，三态门，集电极开路门
- **组合逻辑电路**：组合逻辑电路分析与设计、译码器、数据选择器、加法器、比较器等
- **触发器**：基本RS触发器、D触发器、JK触发器、T触发器
- **时序逻辑电路**：时序电路的分析与设计、计数器、寄存器、移位寄存器等
- **脉冲波形的产生与整形**：555定时器构成的三种典型电路
- **存储器与可编程逻辑器件**：ROM、RAM、与或阵列及可编程器件结构
- **数/模和模/数转换器**：A/D和D/A主要技术指标
- **硬件描述语言**

微机原理

- **数制与编码**：常用数制转换，有符号数运算及溢出规则，BCD码等
- **8086CPU结构与功能**：CPU外部/内部结构、功能、寄存器组织，存储器和I/O等
- **8086CPU指令系统**：算术运算、逻辑运算、移位和标志位操作，循环控制及寻址方式等
- **汇编语言程序设计**：汇编语言程序结构、参数传递方式等
- **总线及其形成**：总线定义及分类，锁存器、触发器、三态门等设计总线
- **存储器设计**：存储器类型、存储器扩展方法等
- **常用芯片的接口技术**：外设接口设计，74373/374/273/245等接口芯片
- **中断系统与中断控制器8259A**：中断概念，8259A编程
- **定时/计数器8253应用设计**：8253内部结构及工作方式、定时与计数应用等
- **并行接口芯片8255A应用设计**：8255A并行接口芯片工作方式与应用

课程背景介绍

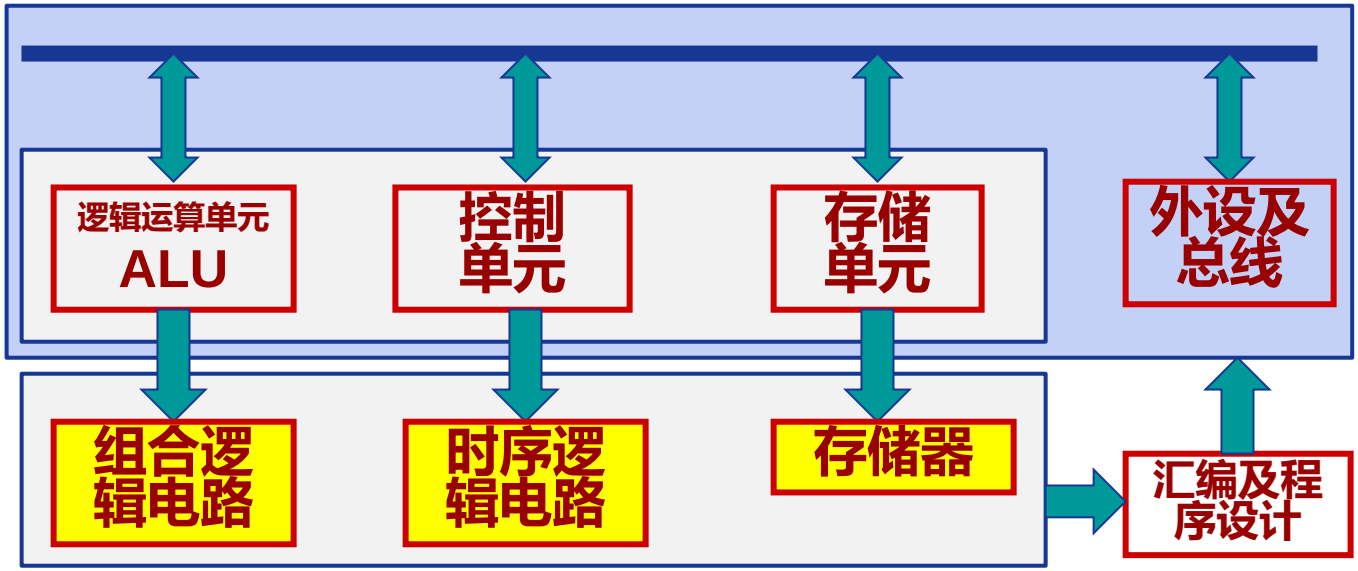
数字逻辑与微处理器课程学时设置

数字逻辑与微处理器

4学分
48+16课时(线下线上混合)

微机原理内容

数字电路内容



（1）MOOC网站【登录爱课程网站搜索“数字电路与系统设计”】：
https://www.icourse163.org/course/XIDIAN-1002203001
（2）学在西电学习网站：
http://mooc1.chaoxing.com/course/222384027.html
（3）微机原理MOOC网站：
http://mooc1-api.learning.xidian.edu.cn/course/1088.html

课程背景介绍

数字逻辑与微处理器课程线上线下混合式教学课程安排

数字逻辑与微处理器技术基础	数字理论基础	数制与编码	（4学分）48学时+16线上MOOC
		逻辑代数基础	
	微处理器结构与数字逻辑	微处理器基本结构与功能	
		组合逻辑电路与运算逻辑单元(ALU)	
		触发器(DFF)结构及功能	
		时序逻辑电路	
	总线与接口外设	集成逻辑门及总线驱动	
		总线及其形成	
		存储器及可编程逻辑器件	
	指令结构及程序设计	CPU设计及指令执行方式	
		CPU指令系统	
		机器语言编码设计	
配套实验	微处理器程序设计实验	微处理器编程（理论）	16学时
		程序设计（实验）	
	数字电路与EDA实验	硬件描述语言（理论）	16学时
		综合设计（实验）	

线下理论48学时，
实验32学时；
线上慕课16学时。

在线课程内容

配套慕课	进制及转换	16学时
	代数法化简	
	TFF、JKFF	
	脉冲波形产生与整形	
	8259A/8253/8255A	
	ADC/DAC	

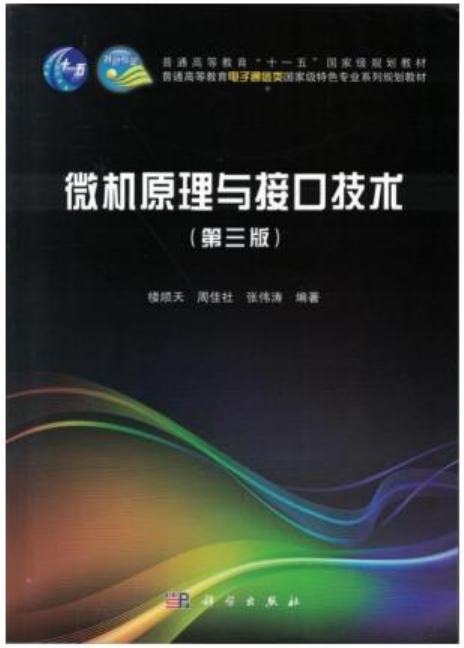
数字逻辑与微处理器

参考教材:

杨颂华，冯毛官，孙万蓉，数字电子技术基础（第三版），西安电子科技大学出版社，2016

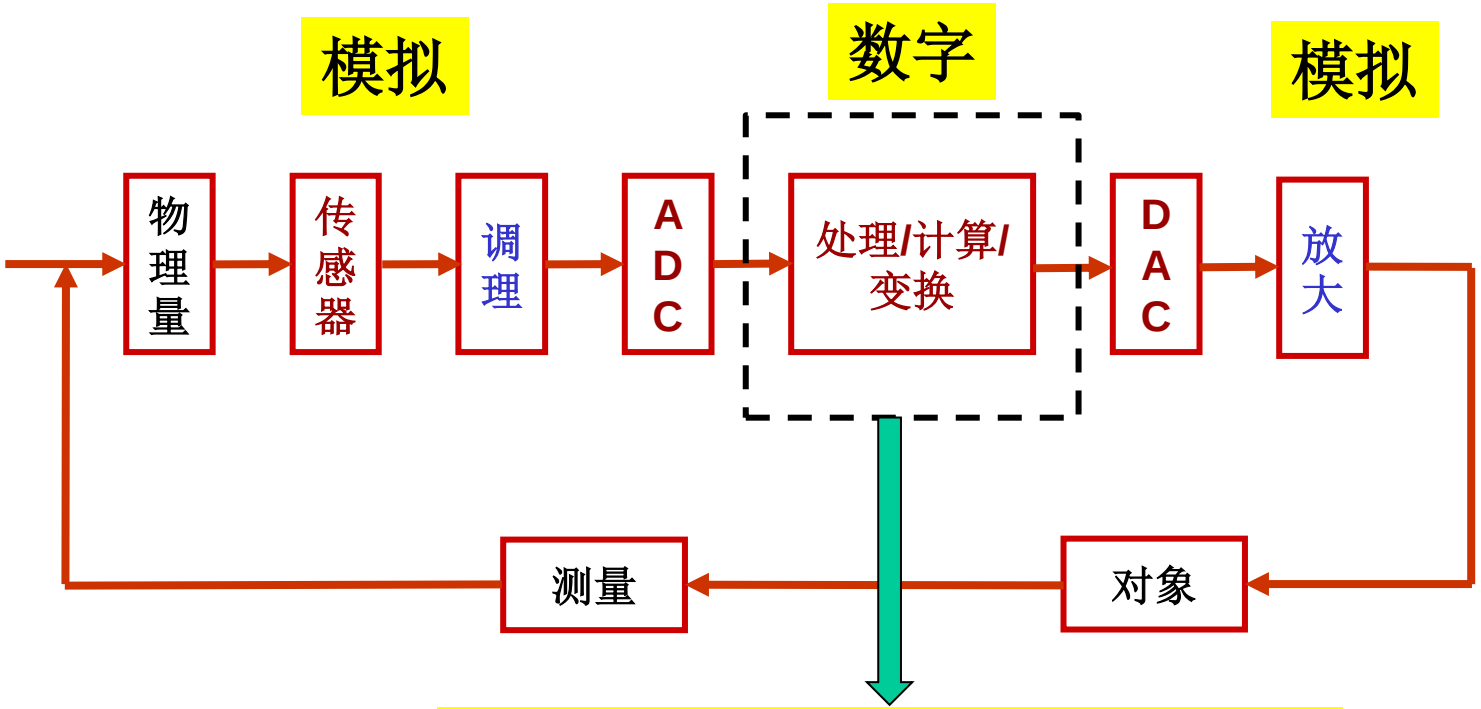
楼顺天，周佳社，张伟涛，微机原理与接口技术（第三版），北京：科学出版社，2020

D.A. Patterson, J.L. Hennessy, Computer Organization and Design (RISC-V Edition), Morgan Kaufmann Publishers, Elsevier 2017.



课程背景介绍

一个通用硬件系统基本结构示例：



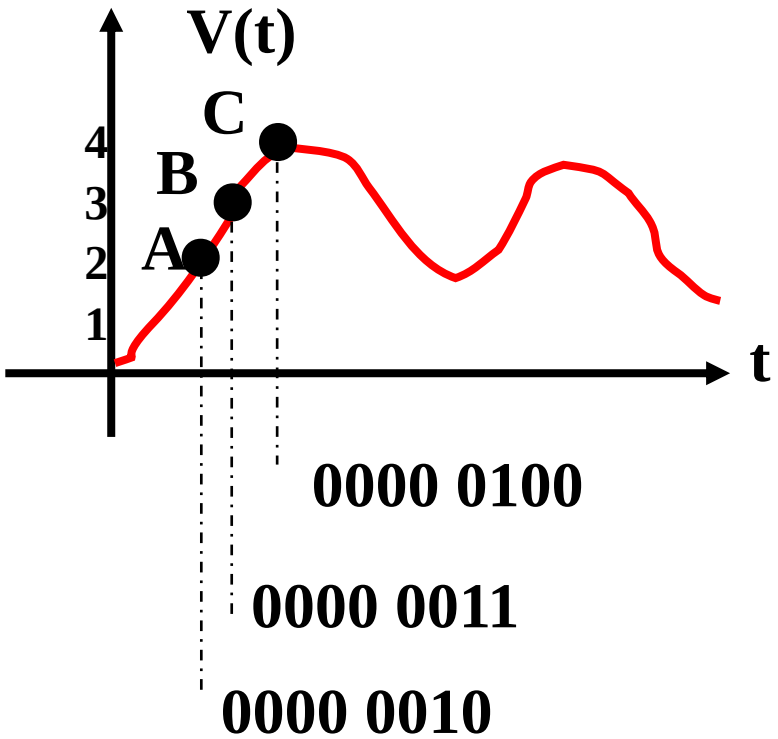
“数字逻辑与微处理器“课程

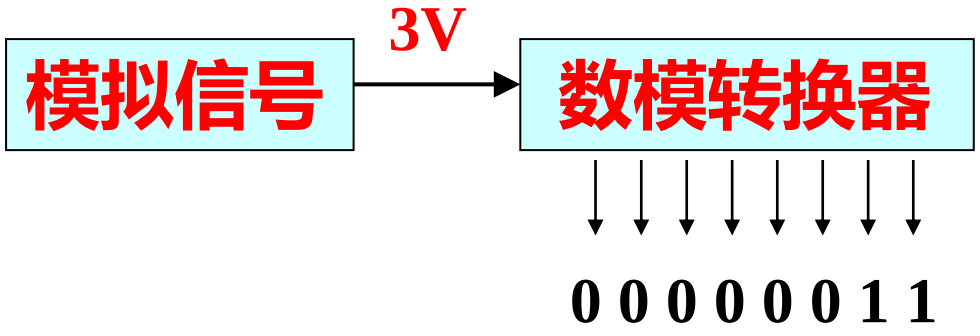
对“数”进行处理/计算/变换

1.1.1 模拟与数字

1.1.1 模拟与数字

模拟量用数字0、1的编码表示（模拟量的数字化）：





如果取样点足够多，原模拟信号可较真实的由对应的数字信号复原。

1.1.2 数字电路的优点

数字系统的优点总结：

(1) 便于高度集成化。

(2) 工作可靠性高、抗干扰能力强。

(3) 数字信息便于长期保存。

(4) 数字集成电路产品系列多、通用性强、成本低。

(5) 保密性好。

数字系统的基本功能就是对数进行加工和处理，包括数的运算、存储和变换等，因此我们首先要对数的基本特征有所了解。

1.2 数 制

如：十进制数　5555

进位计数制中需要理解的三个术语：

（1）**进位制：　进位计数制，简称进位制。**

（2）**基数（底数）：进位制用到的数码个数，即“逢几进一”中的“几”。**

（3）**位权（位的权数）：在某一进位制中，每一位的大小都对应着该位上的数码乘上一个固定的数，这个固定的数就是这一位的权数。权数是一个幂。**

十进制数表示方法：

$$435.86 = 4\times10^{2} + 4\times10^{1} + 5\times10^{0} + 8\times10^{-1} + 6\times10^{-2}$$

上式左边称为**位置记数法或并列表示法**，右边称为**多项式表示法或按权展开法**。

任何一个十进制数N， 都可以用位置记数法和多项式表示法写为：

$$(N)_{10} = a_{n-1}a_{n-2}\cdots a_1a_0\cdot a_{-1}a_{-2}\cdots a_{-m}$$

$$= a_{n-1}\times10^{n-1} + a_{n-2}\times10^{n-2} + \cdots + a_1\times10^{1} + a_0\times10^{0} + a_{-1}\times10^{-1}$$

$$+a_{-2}\times10^{-2} + \cdots + a_{-m}\times10^{-m}$$

$$= \sum_{i=-m}^{n-1} a_i\times10^{i}$$

式中，n代表整数位数，m代表小数位数，$a_i(-m\le i\le n-1)$表示第i位数码，它可以是0、1、2、3、…、9 中的任意一个，10^i为第i位数码的权值。

推广：上述十进制数的表示方法也可以推广到任意进制数。对于一个基数为$R(R\geq 2)$的R进制计数制，数N可以写为

$$(N)_R = a_{n-1}a_{n-2}\cdots a_1a_0\cdot a_{-1}a_{-2}\cdots a_{-m} \qquad 0\leq a_i\leq R-1$$

$$= a_{n-1}\times R^{n-1}+a_{n-2}\times R^{n-2}+\cdots+a_1\times R^1+a_0\times R^0+a_{-1}\times R^{-1}$$

$$+a_{-2}\times R^{-2}+\cdots+a_{-m}\times R^{-m}$$

$$=\sum_{i=-m}^{n-1} a_i R^i$$

式中，n代表整数位数，m代表小数位数，a_i为第i位数码，它可以是0、1、 …、$(R-1)$个不同数码中的任何一个，R^i为第i位数码的权值。

2. 二进制数(Binary)

数码为：0、1；**基数**是2。

运算规律：逢2进1，即：1 + 1 = 10。

如：$(101.01)_2$

$= 1\times2^2 + 0\times2^1 + 1\times2^0 + 0\times2^{-1} + 1\times2^{-2} = (5.25)_{10}$

运算规则

加法规则：0+0=0，0+1=1，1+0=1，1+1=10

乘法规则：0•0=0， 0•1=0 ，1•0=0，1•1=1

3. 八进制(Octal)

数码为：0～7；**基数**是8。

运算规律：逢8进1，即：7 + 1 = 10。

如：$(207.04)_8$

$= 2\times8^2 + 0\times8^1 + 7\times8^0 + 0\times8^{-1} + 4\times8^{-2}$

$= (135.0625)_{10}$

4. 十六进制(Hexadecimal)

数码为：0～9、A～F；**基数**是16。

运算规律：逢16进1，即：F + 1 = 10。

如：$(D8.A)_{16}$

$$= 13\times16^{1} + 8\times16^{0} + 10\times16^{-1}$$

$$= (216.625)_{10}$$

总结： **对于一个基数为$R(R\geq 2)$的R进制计数制，数N可以写为**

$$(N)_R = a_{n-1}a_{n-2}\cdots a_1a_0\cdot a_{-1}a_{-2}\cdots a_{-m} \qquad 0\leq a_i\leq R-1$$

$$= a_{n-1}\times R^{n-1}+a_{n-2}\times R^{n-2}+\cdots+a_1\times R^1+a_0\times R^0+a_{-1}\times R^{-1}$$

$$+a_{-2}\times R^{-2}+\cdots+a_{-m}\times R^{-m}$$

$$=\sum_{i=-m}^{n-1} a_iR^i$$

为什么引入八进制、十六进制?

R=2:	二进制数(Binary)	2^1
R=8:	八进制数(Octal)	2^3
R=16:	十六进制数(Hexadecimal)	2^4
R=10:	十进制数(Decimal)	

1.1.2 进位计数制之间的转换

1. 二进制数与十进制数之间的转换

1) 二进制数转换成十进制数——按权展开法

例如：

$$(10110.11)_2 = 1\times 2^4 + 1\times 2^2 + 1\times 2^1 + 1\times 2^{-1} + 1\times 2^{-2} = (22.75)_{10}$$

方法：二进制数转换成十进制数时，只要将二进制数位置表示法展开成代数式表示，然后将各项按十进制数计算乘法和相加，便可得到等值的十进制数。

同理，若将任意进制数转换为十进制数，只需将数$(N)_R$写成按权展开的多项式表示式，并按十进制规则进行运算，便可求得相应的十进制数$(M)_{10}$。

2) 十进制数转换成二进制数

① 整数转换——除2取余法。

如: $(105)_{10} = (\ ?\ )_2$

		余数	
2	105		
2	52	1	a_0
2	26	0	a_1
2	13	0	a_2
2	6	1	a_3
2	3	0	a_4
2	1	1	a_5
	0	1	a_6

$$(105)_{10} = (1101001)_2$$

2) 十进制数转换成二进制数

② 小数转换——乘2取整法。

如: $(0.4)_{10} = (\ ?\ )_2$

		整数	
	0.4		
×	2		
	0.8	0	a_{-1}
×	2		
	1.6	1	a_{-2}
	0.6		
×	2		
	1.2	1	a_{-3}

小数部分乘2取整的过程，不一定能使最后乘积为0，因此转换值存在误差。

$$(0.4)_{10} \approx (0.011)_2$$

总结：十进制数转换为其它进制数

将一个带有**整数**和**小数**的**十进制数**转换成**二进制**数时，必须将整数部分按**除2**取余法，小数部分按**乘2**取整法进行转换，然后再将两者的转换结果合并起来。

同理，若将**十进制数转换成任意R进制数**，则整数部分转换采用**除R**取余法；小数部分转换采用**乘R**取整法。

2. 二进制数与八进制数、十六进制数之间的相互转换

八进制数和十六进制数的基数R分别为$8=2^3$，$16=2^4$， 所以**三位二进制数恰好相当一位八进制数**，**四位二进制数相当一位十六进制数**， 它们之间的相互转换是很方便的。

R=2:	二进制数(Binary)	2^1
R=8：	八进制数(Octal)	2^3
R=16：	十六进制数(Hexadecimal)	2^4

MOOC学习

假如把一个 9位的二进制整数N转换为8进制数:

$(N)_2=(b_8\,b_7\,b_6\,b_5b_4b_3b_2b_1b_0)_2=(\,?\,)_8$

$$(N)_2=b_8 2^8+b_7 2^7+b_6 2^6+b_5 2^5+b_4 2^4+b_3 2^3+b_2 2^2+b_1 2^1+b_0 2^0$$

$$=(b_8 2^8+b_7 2^7+b_6 2^6)+(b_5 2^5+b_4 2^4+b_3 2^3)+(b_2 2^2+b_1 2^1+b_0 2^0)$$

$$=2^6\underbrace{(b_8 2^2+b_7 2^1+b_6 2^0)}_{o_2}+2^3\underbrace{(b_5 2^2+b_4 2^1+b_3 2^0)}_{o_1}+2^0\underbrace{(b_2 2^2+b_1 2^1+b_0 2^0)}_{o_0}$$

b_i: 1 or 0

o_i: 0 ~ 7

$$=8^2(b_8 2^2+b_7 2^1+b_6 2^0)+8^1(b_5 2^2+b_4 2^1+b_3 2^0)+8^0(b_2 2^2+b_1 2^1+b_0 2^0)$$

$$=8^2 o_2+8^1 o_1+8^0 o_0=(o_2 o_1 o_0)_8$$

(shorthand notation)

MOOC学习

假如把一个6位的二进制小数N转换为8进制数:

$(N)_2=(b_{-1}\,b_{-2}\,b_{-3}\,b_{-4}b_{-5}b_{-6})_2=(\,?\,)_8$

$$(N)_2=b_{-1}2^{-1}+b_{-2}2^{-2}+b_{-3}2^{-3}+b_{-4}2^{-4}+b_{-5}2^{-5}+b_{-6}2^{-6}$$

$$=(b_{-1}2^{-1}+b_{-2}2^{-2}+b_{-3}2^{-3})+(b_{-4}2^{-4}+b_{-5}2^{-5}+b_{-6}2^{-6})$$

$$=2^{-3}\underbrace{(b_{-1}2^{2}+b_{-2}2^{1}+b_{-3}2^{0})}_{o_{-1}}+2^{-6}\underbrace{(b_{-4}2^{2}+b_{-5}2^{1}+b_{-6}2^{0})}_{o_{-2}}$$

b_i: 1 or 0

o_i: 0 ~ 7

$$=8^{-1}(b_{-1}2^{2}+b_{-2}2^{1}+b_{-3}2^{0})+8^{-2}(b_{-4}2^{2}+b_{-5}2^{1}+b_{-6}2^{0})$$

$$=8^{-1}o_{-1}+8^{-2}o_{-2}=(o_{-1}o_{-2})_8$$

(shorthand notation)

2. 二进制数与八进制数、十六进制数之间的相互转换（总结）

八进制数和十六进制数的基数分别为 $8=2^3$，$16=2^4$

1）2进制数转换为8进制、16进制数

三（四）位一组，不足左补零

三（四）位一组，不足右补零

小数点

2）8进制、16进制数转换为2进制数

8进制数 ⟶ 2进制数：1位变3位

16进制数 ⟶ 2进制数：1位变4位

例 1: $(110101.011000111)_2 = (\quad ? \quad)_8 = (\quad ? \quad)_{16}$

$$\underline{110}\ \underline{101}\ .\ \underline{011}\ \underline{000}\ \underline{111}$$
$$6\quad 5\ .\ 3\quad 0\quad 7$$
$$\underline{0011}\ \underline{0101}\ .\ \underline{0110}\ \underline{0011}\ \underline{1000}$$
$$3\quad 5\ .\ 6\quad 3\quad 8$$

$$(110101.011000111)_2 = (65.307)_8 = (35.638)_{16}$$

例 2: $(2EA)_{16} = (\quad ? \quad)_{10}$

$$2\quad E\quad A$$
$$\underline{0010}\ \underline{1110}\ \underline{1010}$$

$$(0010111010 10)_2 = 512+128+64+32+10 = (746)_{10}$$

$$(2EA)_{16} = (746)_{10}$$

1.2 编 码

用一定位数的二进制数来表示十进制数码、字母、符号等信息称为**编码**。

这一定位数的二进制数就称为**代码**。

对于N个信息，要用几位二进制数才能满足编码呢?

$$2^n \geq N$$

1.2 编 码

1.2.1 二—十进制编码(BCD码)

二—十进制编码是用四位二进制码的10 种组合表示十进制数0~9，简称BCD码(Binary Coded Decimal)。

$$A_{16}^{10} = \frac{16!}{(16-10)!} \approx 2.9 \times 10^{10}$$

1.2.1 二—十进制编码(BCD码)

★• 8421 BCD 码： **有权码，且编码唯一！**

$$(a_3a_2a_1a_0)_{8421BCD}=8\times a_3+4\times a_2+2\times a_1+1\times a_0$$

• 5421 BCD 码： **有权码，且编码不唯一！**

$$(a_3a_2a_1a_0)_{5421BCD}=5\times a_3+4\times a_2+2\times a_1+1\times a_0$$

• 2421 BCD 码： **有权码，且编码不唯一！**

$$(a_3a_2a_1a_0)_{2421BCD}=2\times a_3+4\times a_2+2\times a_1+1\times a_0$$

★• 余3码 码： **无权码，且编码唯一！**

$$(b_3b_2b_1b_0)_{余3码}=(a_3a_2a_1a_0)_{8421BCD}+(0011)_2$$

表 1-2 几种常用的 BCD 码

Decimal	8421 BCD	5421 BCD	2421 BCD	Xs3 Code
0	0 0 0 0	0 0 0 0	0 0 0 0	0 0 1 1
1	0 0 0 1	0 0 0 1	0 0 0 1	0 1 0 0
2	0 0 1 0	0 0 1 0	0 0 1 0	0 1 0 1
3	0 0 1 1	0 0 1 1	0 0 1 1	0 1 1 0
4	0 1 0 0	0 1 0 0	0 1 0 0	0 1 1 1
5	0 1 0 1	1 0 0 0	1 0 1 1	1 0 0 0
6	0 1 1 0	1 0 0 1	1 1 0 0	1 0 0 1
7	0 1 1 1	1 0 1 0	1 1 0 1	1 0 1 0
8	1 0 0 0	1 0 1 1	1 1 1 0	1 0 1 1
9	1 0 0 1	1 1 0 0	1 1 1 1	1 1 0 0
unused	1 0 1 0	0 1 0 1	0 1 0 1	0 0 0 0
	1 0 1 1	0 1 1 0	0 1 1 0	0 0 0 1
	1 1 0 0	0 1 1 1	0 1 1 1	0 0 1 0
	1 1 0 1	1 1 0 1	1 0 0 0	1 1 0 1
	1 1 1 0	1 1 1 0	1 0 0 1	1 1 1 0
	1 1 1 1	1 1 1 1	1 0 1 0	1 1 1 1

任何一个十进制数均可以用BCD码表示，如：

$$432.25=(0100\ 0011\ 0010.0010\ 0101)_{8421BCD}$$

$$=(0111\ 0110\ 0101.0101\ 1000)_{余3}$$

$$=(\qquad ?\qquad)_2$$

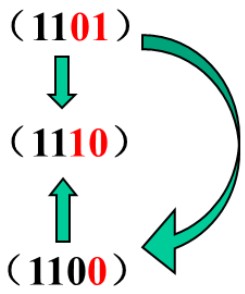
1.2.2 可靠性编码

1. Gray码(格雷码)

Gray码也称<u>循环码</u>，其最基本的特性是任何相邻的两组代码中，仅有一位数码不同，因而又叫<u>单位距离码</u>。

Decimal	$B_3\ B_2\ B_1\ B_0$	$G_3\ G_2\ G_1\ G_0$
0	0 0 0 0	0 0 0 0
1	0 0 0 1	0 0 0 1
2	0 0 1 0	0 0 1 1
3	0 0 1 1	0 0 1 0
4	0 1 0 0	0 1 1 0
5	0 1 0 1	0 1 1 1
6	0 1 1 0	0 1 0 1
7	0 1 1 1	0 1 0 0
8	1 0 0 0	1 1 0 0
9	1 0 0 1	1 1 0 1
10	1 0 1 0	1 1 1 1
11	1 0 1 1	1 1 1 0
12	1 1 0 0	1 0 1 0
13	1 1 0 1	1 0 1 1
14	1 1 1 0	1 0 0 1
15	1 1 1 1	1 0 0 0

单位距离特性



典 型 的 格 雷 码 写 法

两位格雷码	三位格雷码	四位格雷码
0 0	0 0 0	0 0 0 0
0 1	0 0 1	0 0 0 1
1 1	0 1 1	0 0 1 1
1 0	0 1 0	0 0 1 0
	1 1 0	0 1 1 0
	1 1 1	0 1 1 1
	1 0 1	0 1 0 1
	1 0 0	0 1 0 0
		1 1 0 0
		1 1 0 1
		1 1 1 1
		1 1 1 0
		1 0 1 0
		1 0 1 1
		1 0 0 1
		1 0 0 0

低位镜像，高位取反

MOOC学习

2. 奇偶校验码 (Odd/Even codes）

代码(或数据)在传输和处理过程中，有时会出现代码中的某一位由 0 错变成 1，或 1 变成 0。奇偶校验码是一种具有检验出这种错误的代码，奇偶校验码由信息位和一位奇偶检验位两部分组成。

信息位：是位数不限的任一种二进制代码。

检验位：仅有一位，它可以放在信息位的前面，也可以放在信息位的后面。

MOOC学习

2、奇偶校验码 (Odd/Even codes)

编码方法：

信息位	检验位

使得一组代码中信息位和检验位中“1”的个数之和为奇数，称为**奇检验**（odd codes）；

使得一组代码中信息位和检验位中“1”的个数之和为偶数，称为**偶检验**（Even codes）。

MOOC学习

表 1-4 带奇偶检验的8421 BCD码

十进制数	8421 BCD 奇检验		8421 BCD 偶检验	
0	0000	1	0000	0
1	0001	0	0001	1
2	0010	0	0010	1
3	0011	1	0011	0
4	0100	0	0100	1
5	0101	1	0101	0
6	0110	1	0110	0
7	0111	0	0111	1
8	1000	0	1000	1
9	1001	1	1001	0
	信息位	检验位	信息位	检验位

字符的编码

ASCII —— **美国国家信息标准交换码**

ASCII → **用7位二进制代码对任一字符编码，包括：**

共128个
- **32个通用控制符**
- **0--9 10个数字**
- **52个英文大小写字母**
- **34个专用符号**

要求掌握常用字符的ASCII码：

● **ASCII：** 0～9(30H~39H) A~Z(41H~5AH) a~z(61H~7AH)

LF(换行) 0A, CR(回车) 0D, ESC 1BH, SP(空格) 20H

字符的编码

表 1-5 ASCII码

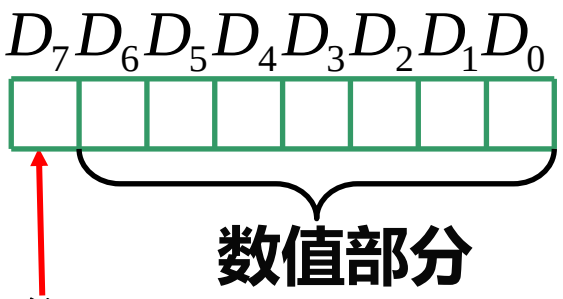
	$B_4B_3B_2B_1$ \ $B_7B_6B_5$	0	1	2	3	4	5	6	7
		0	0	0	0	1	1	1	1
		0	0	1	1	0	0	1	1
		0	1	0	1	0	1	0	1
0	0 0 0 0	NUL	DLE	Sp	0	@	P	`	p
1	0 0 0 1	SOH	DC1	!	1	A	Q	a	q
2	0 0 1 0	STX	DC2	"	2	B	R	b	r
3	0 0 1 1	ETX	DC3	#	3	C	S	c	s
4	0 1 0 0	EOT	DC4	$	4	D	T	d	t
5	0 1 0 1	ENQ	NAK	%	5	E	U	e	u
6	0 1 1 0	ACK	SYN	&	6	F	V	f	v
7	0 1 1 1	BEL	ETB	'	7	G	W	g	w
8	1 0 0 0	BS	CAN	(	8	H	X	h	x
9	1 0 0 1	HT	EM	)	9	I	Y	i	y
A	1 0 1 0	LF	SUB	*	:	J	Z	j	z
B	1 0 1 1	VT	ESC	+	;	K	[	k	{
C	1 1 0 0	FF	FS	,	<	L	\	l	|
D	1 1 0 1	CR	GS	—	=	M	]	m	}
E	1 1 1 0	SO	RS	.	>	N	^	n	~
F	1 1 1 1	SI	US	/	?	O	—	o	DEL

有符号数的表示

1. 符号的表示方法

为了表示一个有符号数，除了数值位以外，还应制定**符号位**，通常以这个数的最高位表示符号位。

我们假定讨论的数为整数，对8位有符号二进制整数，用下表示：



0：表示正数
1：表示负数
符号位

有符号数的表示

2. 原码表示法

约定数码序列中的最高位为符号位；
正数的符号位用0表示，负数的符号位用1表示；
其余有效数值部分则用二进制的绝对值表示。

一个数X的原码记为：[X]原

D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
0	0	0	0	0	0	1	1

+3的表示 [+3] 原 =00000011B

数值部分=绝对值(用无符号二进制数表示)

符号位

有符号数的表示

2. 原码表示法

1	0	0	0	0	0	1	1

-3的表示 [-3]原 =10000011B

[+0]原 =0 0 0 0 0 0 0 0B　　[-0]原=1 0 0 0 0 0 0 0B

数0的两种表示方法

对8位有符号二进制数**用原码表示**的范围：

正数从00000000～01111111，+0～+127

负数从10000000～11111111，-0～-127

有符号数的表示

2. 原码表示法

有符号二进制数用原码表示的优缺点：

优点：表示简单，易于理解，真值转换方便。

缺点：+、-运算麻烦。因为它仅仅是将其值的符号用一位二进制数表示，因而它的原码数的+、-运算完全同笔算。如两个数相加，计算机首先要对其符号进行判断,如果同号,则进行相加运算,如果异号,则进行相减运算,最后在结果的前面加上正确的正负号。

势必增加运行时间，降低速度，使运算器的逻辑复杂化。为了改进它，引进了补码的概念。

有符号数的表示

3. 补码表示法

（1）补码的定义

一个数X的补码记为 $[X]_{补}$ 补码可定义为：

$$[x]_{补}=\begin{cases} x & 当\quad 0\le x<2^{n-1} \\ 2^n+x & 当\quad -2^{n-1}\le x<0 \end{cases} \pmod{2^n}$$

从定义可见，正数的补码=原码，即

$[x]_{补} = [x]_{原}$

所以，只有负数求补的问题。

有符号数的表示

3. 补码表示法

（2）一个数的补码的求法

根据定义求补码

$$[x]_{补} = 2^n + x = 2^n - |x| \quad , x<0$$

即负数x的补码等于模 2^n加上其真值（或减去其真值的绝对值）。

如：x=-1010111B，n=8，则

$$[X]_{补} = 2^8 + (-1010111B)$$

$$= 100000000B - 1010111B$$

$$= 10101001B \quad (\mathrm{mod}\ 2^n)$$

这种方法因要做一次减法，很不方便。

有符号数的表示

3. 补码表示法

利用原码求补码

一个负数的补码等于其原码除符号位保持不变外，其余各位按位取反，再在最低位加1。

如：x=-1010111B　[X]原=11010111B

[X]补= 10101000B+1=10101001B

值的注意的是：0的补码只有唯一的形式，符号位和数值位均为0。无正负0之分。

有符号数的表示

3. 补码表示法

简便的直接求补码

分2步：

第一步：取x的绝对值

第二步：

当x为正数时，$[x]_{补} = |x|_{原}$

当x为负数时，$[x]_{补} = \overline{|x|_{原}} + 1$ （含符号位）

如：x=-87, 即|x|=0101 0111B

$|x|_{原}$ = 0 1 0 1 0 1 1 1 B

1 0 1 0 1 0 0 0 B

+) 1

$[x]_{补}$ = 1 0 1 0 1 0 0 1 B

有符号数的表示

3. 补码表示法

(3) 补码的运算规则

第一个公式： 两个n位二进制数之和的补码等于这两数补码之和，即：

$$[X+Y]_{补} = [X]_{补} + [Y]_{补}$$

例： (+33) + (-15)

	0 0 1 0 0 0 0 1 B	$[+33]_{补}$
	+ 1 1 1 1 0 0 0 1 B	$[-15]_{补}$
进位，丢掉 →	[1]0 0 0 1 0 0 1 0 B	$[+18]_{补}$

有符号数的表示

3. 补码表示法

（3）补码的运算规则

第二个公式： 两个n位二进制数之差的补码等于这两数补码之差，即：

$$[X-Y]_{补} = [X]_{补} - [Y]_{补}$$

例：（+33）-（-15）

	0 0 1 0 0 0 0 1 B	$[+33]_{补}$
	- 1 1 1 1 0 0 0 1 B	$[-15]_{补}$
借位，丢掉 →	[1]0 0 1 1 0 0 0 0 B	$[+48]_{补}$

有符号数的表示

3. 补码表示法

（3）补码的运算规则

第三个公式： 补码减法运算时，也可以利用加法基本公式，即：

$$[X-Y]_{补} = [X]_{补} + [-Y]_{补} \pmod{2^n}$$

因为：$X-Y = X+(-Y)$

所以：$[X-Y]_{补} = [X+(-Y)]_{补} = [X]_{补} + [-Y]_{补}$

一般称已知 $[Y]_{补}$，求得 $[-Y]_{补}$ 的过程叫变补或求负

有符号数的表示

3. 补码表示法

（3）补码的运算规则

变补或求负是一种很有用的运算。求法：

若已知 [Y]补 $Y_1Y_0 \ldots\ldots Y_{n-1}Y_{n-2}$ ，则对 [Y]补 的每一位（包括符号位）都按位取反，然后再加1， 结果即 [-Y]补

例： （+33）- （+15） ⟺ （+33）+ （-15）

00100001B [+33]补	00100001B [+33]补
- 00001111B [+15]补	+ 11110001B [-15]补
00010010B [+18]补	[1]00010010B [+18]补

借位，丢掉

有符号数的表示

3. 补码表示法

(4) 有符号数运算的溢出问题

如果计算机的字长为n位，n位二进制数的最高位为符号位，其余n-1位为数值位，采用补码表示法时，可表示的数X的范围为：

$$-2^{n-1} \le X \le 2^{n-1} - 1$$

当n=8时，可表示的有符号数的范围为：

-128 ～ +127

当n=16时，可表示的有符号数的范围为：

-32768 ～ +32767

有符号数的表示

3. 补码表示法

（4）有符号数运算的溢出问题

两个有符号数进行加减运算时，如果运算结果超出可表示的有符号数的范围时，就会发生溢出，使计算结果出错。

很显然，溢出只能出现在两个同号数相加或两个异号数相减的情况下。

例： （+72）+（+98）=+170>+127　　溢出

```
  0 1 0 0 1 0 0 0 B [+72]补
+ 0 1 1 0 0 0 1 0 B [+98]补
  1 0 1 0 1 0 1 0 B [-86]补
```

有进位 C_p =1

无进位 C_S =0

溢出，结果出错（正溢出）

$$C_s \forall C_p = 1$$

设次高位（数值部分最高位）向最高位（符号位）的进位标志为 C_p；最高位（符号位）和次高位的进位相加的进位标志为 C_S。

例： （-83）+（-80）=-163<-128　　溢出

```
  1 0 1 0 1 1 0 1 B [-83]补
+ 1 0 1 1 0 0 0 0 B [-80]补
[1]0 1 0 1 1 1 0 1 B [+93]补
```

无进位 C_p=0

有进位 C_S=1

溢出，结果出错（负溢出）

$$C_s \forall C_p = 1$$

有符号数的表示

3. 补码表示法

(4) 有符号数运算的溢出问题

结论：对于加法运算

(1) 如果次高位有进位而最高位无进位，则结果溢出；
(2) 如果次高位无进位而最高位有进位，则结果溢出。

同理：对于减法运算

(1) 如果次高位有借位而最高位无借位，则结果溢出；
(2) 如果次高位无借位而最高位有借位，则结果溢出。

这两种情况分别是：

(1) 两负数相加，结果超出范围，形式上变为正数；

(2) 两正数相加，结果超出范围，形式上变为负数。

浮点数的表示

一、浮点数表示原理

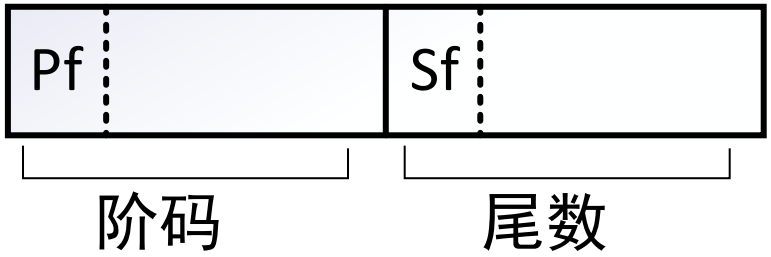
十进制：$65.241=0.65241*10^{2}=65241*10^{-3}=\cdots$

二进制：$100.101B=0.100101B*2^{3}=1001.01B*2^{-1}=\cdots$

任意一个二进制实数N可表示为：$N=S*2^{P}$

S：尾数，S的符号代表实数N的正负，S的值表示N的有效值

P：阶码，P的符号代表小数点移动的方向，P的值移动的位数



浮点数的表示

浮点数规格化操作：

约束S的真值为0.5~1（也就是纯小数的小数点后第一位必须为1）

例：将实数-12.375表示为规格化的浮点数

$$-12.375=-1100.011B=-0.1100011B*2^{100B}$$

若用4位有符号二进制数表示阶码，用8位有符号数表示尾数，均用原码 则有

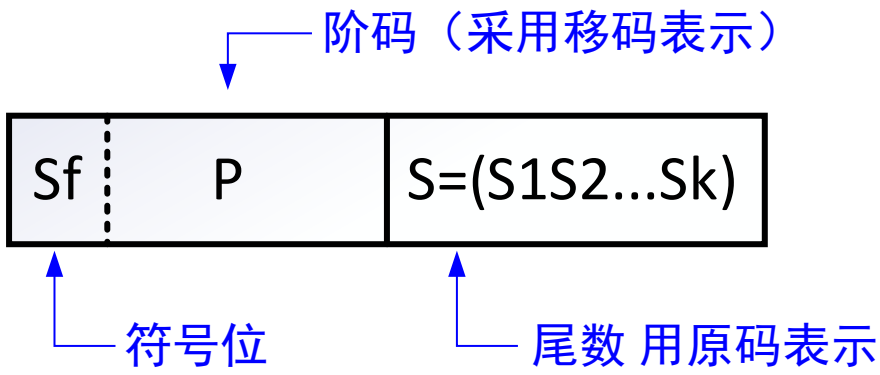
小数点隐含

0 1 0 0 1 . 1 1 0 0 0 1 1

Pf　　Sf

浮点数的表示

一、标准化浮点数



参数	单精度浮点
字长	32
符号位Sf长度	1
尾数S长度K	23
阶码P长度	8
阶码范围	1~254
指数偏移量Q	127

实数N的标准化浮点表示为：

$$N=(-1)^{Sf}*0.(S_0S_1S2\ldots S_K)*2^{P-Q}$$

- **S0固定为1，无须存储**
- **32位浮点数阶码P占8位，为无符号数等于实际指数加偏移量Q（127），阶码全为1表示无限大，全为0 表示非规格化数**

浮点数的表示

例：将十进制实数-139.8125表示为单精度浮点数

-139.8125=-10001011.1101B

=-0.100010111101B*2^8

符号位：Sf=1

阶码：　P=8+127=135=10000111B

尾数:S=0001 0111 1010 0000 0000 000B(注：不含S_0)

单精度浮点表示结果为：

1	1000 0111	0001 0111 1010 0000 0000 000
Sf	P	S=(S1S2…S23)